Proposed changes to "Where the items are", version 3

10 November 2026

How to read this

Part 1 lists the changes in plain words. Part 2 shows the Problem Formulation section as it would read. Part 3 shows what changes in the Implementation chapter and Appendix C. Text in green is proposed and not in the thesis yet, and ~~struck~~ text would go. Everything in black is already in the thesis file. Part 4 lists what the check of the code found and the questions for you.

Part 1. The changes

The idea

The problem is stated the way Waubert de Puiseau et al. (2022) state theirs. It has a decision, what may be seen, what may not be done, and a cost. How the problem is solved is separate. Here it is solved by three rules, and a rule does not break the storage location assignment problem. It is one way to solve it, as are the rule benchmarks in their paper and the classic slotting rule. The only thing a rule may not claim is to be the best possible placement.

The real cost is the mean order service time, the main measure of the thesis. Until now the section gave $D + \beta K$ as the objective, but that is only the stand-in the rules rank shelves by. So D, K and β move to the Implementation chapter, where the rules are.

Changes in the Problem Formulation (Part 2)

- **How much arrives.** New sentences say that stock comes in refills of Q units, that a refill goes whole onto one shelf, and that a SKU gets one refill for every Q units ordered. This was missing, and it is what keeps the arrivals the same under every rule.
- **A new example and Figure ??**, with three refills of 3 units of tea. Two are placed and one fits nowhere, which also shows why condition (**??**) uses $\leq$.
- **"What the rule decides" becomes "The storage problem"**. One sentence names the problem and its four parts (decision, what is seen, the conditions, the cost) without repeating them. A second paragraph names the three rules in one sentence each and says plainly that congestion-aware storage is the thesis's own, because the DSLAP studies have no traffic.
- **D, K, β and the objective leave this section.** They were put here earlier today and would now move (Part 3). α stays dropped.

Changes in the Implementation chapter (Part 3)

- **The stand-in cost** $D+\beta K$ moves here as a new paragraph, with the label `eq:storageobjective`. D now follows the code. Each leg of the trip counts as often as it is made, the pick leg once per order line and the put-away leg once per refill. The sentence that gave both legs the same weight goes.
- **Ties** go by the trip distance, then by index, as in the code.
- **Appendix C's arrivals** are rewritten to the refill rule the code uses, with a new symbol $\nu_t(k)$ for the units ordered since the last refill.
- **Other places** name the crowding cost in words, not as an equation of a later chapter.
- **Whole refills.** The sentence "as many on each shelf as fit" becomes "each refill whole on the first shelf with room for it", as the code does.

Part 2. The Problem Formulation section as it would read

V-C Where the items are

Stock is kept on shelves, and an item can only be picked from a shelf that holds it. So where the stock sits decides where the agents must move to. The shelves never move. What changes over time is what each shelf holds.

The storage state is a counting function. It says how many units of each SKU sit on each shelf, for example 12 units of tea on shelf A. A shelf can hold several SKUs, and one SKU can sit on several shelves.

Formally, $x_t(k, v)$ is the number of units of SKU k on shelf v at time t. Here k is one of the SKUs in $\mathcal{K}$, and v one of the shelves in V_{str}:

$$x_t : \mathcal{K} \times V_{\text{str}} \to \mathbb{N}_0. \tag{5}$$

Each shelf has limited room, and units of different SKUs take different amounts of it. The units on a shelf, each counted with the room it takes, may not need more room than the shelf has. Formally, let b_k be the room one unit of SKU k takes and $\text{cap}(v)$ the room on shelf v. The storage states that fit are

$$\mathcal{X} = \left\{ x \in \mathbb{N}_0^{\mathcal{K} \times V_{\text{str}}} : \sum_{k \in \mathcal{K}} b_k x(k, v) \leq \text{cap}(v) \quad \forall v \in V_{\text{str}} \right\}. \tag{6}$$

Example. Table **??** shows a small storage state. Tea and mugs take one unit of room each, and coffee takes two. Shelves A, B and C have room for 20, 15 and 10. Shelf A is then exactly full, and B and C have room left:

$$\begin{pmatrix} 1 & 2 & 1 \end{pmatrix} \begin{pmatrix} 12 & 0 & 5 \\ 4 & 3 & 0 \\ 0 & 6 & 2 \end{pmatrix} = \begin{pmatrix} 20 & 12 & 7 \end{pmatrix} \leq \begin{pmatrix} 20 & 15 & 10 \end{pmatrix}.$$

Room also ties SKUs together. Tea and coffee share the room on shelf A, so more tea there leaves less room for coffee.

Table 1: A small storage state.

	A	B	C
tea	12	0	5
coffee	4	3	0
mugs	0	6	2

How stock changes. Stock leaves the shelves when agents pick it for orders, and new stock comes in to refill them.

New stock arrives at a dock every Δ timesteps. Each of these moments is a storage update. Stock arrives in refills of Q units, and a refill always goes whole onto one shelf. How much arrives depends on the orders alone. A SKU gets one refill for every Q units ordered since its last refill, so the arrivals are the same under every rule. At a storage update, the storage rule F chooses a shelf for each arriving unit. An agent then carries the units from the dock to that shelf. This trip is called a put-away.

While the units are on their way, they already take up room on the shelf, but no order can pick them yet. So they are counted on their own. Write $x_t^{\text{in}}(k, v)$ for the units of SKU k on their way to shelf v, with "in" for inbound. When the agent delivers them, they count as on the shelf.

What the rule knows. The rule only knows the last storage period. It reads three averages per step over that period:

- $\hat{\rho}_t(k)$, how often SKU k was ordered
- $\hat{\mu}_t(e)$, how often agents crossed edge e
- $\hat{w}_t(e)$, how many steps agents waited to get onto edge e

Demand storage uses only the first, the orders. Congestion-aware storage uses all three. The last two come from how the agents moved, and show which aisles were crowded. An edge counts as crowded when many agents cross it and they often wait to get onto it.

The storage update. At a storage update the rule only decides where the new units go, and three conditions must hold. No shelf may hold more than it has room for, counting the units on their way. Units already on their way keep the shelf they were given. And the rule cannot place more units than arrived, so units that fit on no shelf are left unplaced.

Formally, the rule takes the stock just before the update, x_t^- on the shelves and $x_t^{\text{in},-}$ on its way. It also takes $y_t(k)$, the units of each SKU k that arrived, and the three averages above. It returns where the units go:

$$x_t^{\text{in}} = F\left(x_t^-,\, x_t^{\text{in},-},\, y_t,\, \hat{\rho}_t,\, \hat{\mu}_t,\, \hat{w}_t\right). \tag{7}$$

The three conditions, in the same order, are

$$x_t^- + x_t^{\text{in}} \in \mathcal{X} \qquad \text{no shelf over its room,} \tag{8}$$

$$x_t^{\text{in}} \geq x_t^{\text{in},-} \qquad \text{units on their way keep their shelf,} \tag{9}$$

$$\sum_{v \in V_{\text{str}}} \left(x_t^{\text{in}}(k, v) - x_t^{\text{in},-}(k, v)\right) \leq y_t(k) \quad \forall k \in \mathcal{K} \qquad \text{no more placed than arrived.} \tag{10}$$

Example. Take the stock of Table **??** just before an update, with nothing on its way, and say three refills of 3 units of tea arrive. Shelf A is full, and B and C each have room for one refill, so the rule puts one refill on B and one on C (Figure **??**). Every shelf stays within its room, so (**??**) holds. Nothing was on its way before, so (**??**) holds. The third refill fits nowhere and is left unplaced. The rule places 6 of the 9 units, so (**??**) holds with $6 \leq 9$.

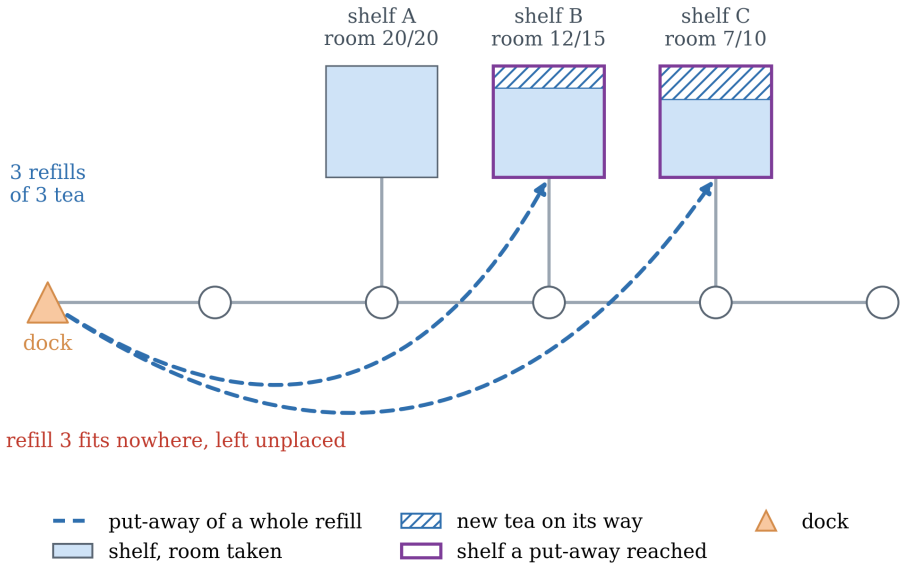


Figure 1: One storage update on the stock of Table **??**. Two refills of tea go whole to shelves with room, and the third fits nowhere.

The storage problem. Choosing a shelf when stock arrives is the dynamic storage location assignment problem (Waubert de Puiseau et al., 2022). The rule makes the storage update (**??**), sees only the past, must keep conditions (**??**) to (**??**), and is judged by the mean order service time of Section V-J.

That cost depends on how the agents move after the update, so no rule can work it out when it decides. Each rule therefore ranks the shelves by a simple stand-in for it, given in Section VI-A. Fixed storage sends every refill to its SKU's home shelf. Demand storage puts popular SKUs where their trip is short, the classic slotting rule (Heskett, 1963; de Koster et al., 2007). Congestion-aware storage also avoids shelves whose trip runs through crowded aisles. The dynamic storage studies have no traffic, so this last rule is this thesis's own, and it is the one that closes the loop.

Example. Figure **??** shows tea that has become popular. Under fixed storage, new tea goes back to its home shelf, three edges from the workstation. Under demand storage, it goes to the nearest shelf with room, one edge away. The old tea stays where it is and runs down as it is picked.

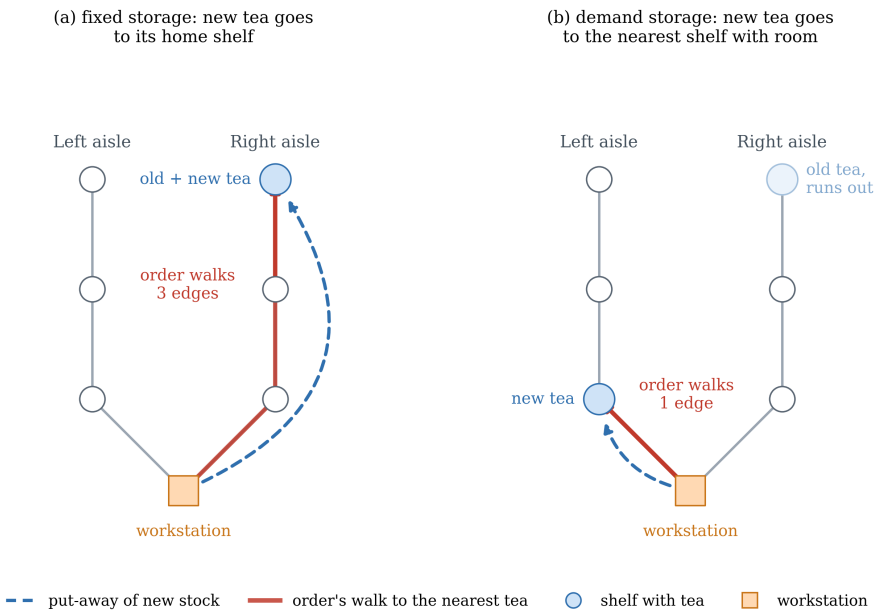


Figure 2: Arriving tea under fixed and demand storage. Old tea stays where it is, so only the new tea's shelf differs.

Part 3. The Implementation chapter, the part that changes

VI-A Storage rules (excerpt)

All three rules place arriving stock the same way, and only their order differs. They take one SKU at a time and put its arriving ~~units on the shelves in the rule's order, as many on each shelf as fit~~ refills on the shelves in the rule's order, each refill whole on the first shelf with room for it. What fits nowhere is not placed. Stock already on a shelf stays where it is.

(The trip distance paragraph, fixed storage and the start of demand storage stay as they are.)

In the congestion-aware paragraph: ~~Both legs carry the same $\hat{\rho}_t(k)$, because arrivals refill what was ordered, so a SKU is put away about as often as it is picked.~~

The stand-in cost. The rules cannot see the service time they are judged by, so they rank shelves by a stand-in. The classic slotting cost is each trip's length times how often the trip is made (Heskett, 1963; de Koster et al., 2007). A shelf has two trips. The pick leg runs from the shelf to a workstation of the SKU's section and is made about $\hat{\rho}_t(k)$ times per step, once per order line. The put-away leg runs from a dock to the shelf and is made about $\hat{\rho}^{\text{unit}}_t(k)/Q$ times per step, because one refill brings Q units.

Formally, let $d^{\text{pick}}_t(k, v)$ be the mean distance from shelf v to the workstations of section $\hat{h}_t(k)$, $d^{\text{put}}(v)$ the mean distance from the docks to v, and $x^{\text{new}}_t = x^{\text{in}}_t - x^{\text{in},-}_t$ the units placed at an update. The slotting cost is

$$D = \sum_{k \in \mathcal{K}} \sum_{v \in V_{\text{str}}} \left(\hat{\rho}_t(k)\, d^{\text{pick}}_t(k, v) + \frac{\hat{\rho}^{\text{unit}}_t(k)}{Q}\, d^{\text{put}}(v) \right) x^{\text{new}}_t(k, v).$$

The crowding cost is this thesis's own. It adds up the crowding $\hat{\mu}_t(e)\, \hat{w}_t(e)$ on every edge e of the trip through v, from the docks to v and on to the nearest workstation,

$$K = \sum_{k \in \mathcal{K}} \sum_{v \in V_{\text{str}}} \Big(\sum_{e \in \text{path}(v)} \hat{\mu}_t(e)\, \hat{w}_t(e) \Big) x^{\text{new}}_t(k, v).$$

Demand and congestion-aware storage aim for a small value of

$$D + \beta K,$$

where β sets how much crowding counts against distance, and demand storage has $\beta = 0$. They are greedy. They place one SKU at a time on its best shelves with room, and they do not search for the smallest value. Shelves that score the same go by the trip distance $d^{\text{trip}}_t(k, v)$, then by index.

Appendix C, the arrivals (rewritten)

How much arrives is set by the orders alone, so it is the same under every rule. Stock comes in refills of Q units. A SKU gets one refill for every Q units ordered since its last refill, and units that do not make a whole refill count towards the next one. So if 23 units of tea were ordered since its last refill and $Q = 10$, two refills of tea arrive and 3 units carry over. Formally, with $\nu_t(k)$ the units of SKU k ordered since its last refill, counted when their lines are released, the arrivals are

$$y_t(k) = Q \left\lfloor \frac{\nu_t(k)}{Q} \right\rfloor,$$

and $\nu_t(k) - y_t(k)$ carries over to the next update.

The old text, which fills each SKU up to a level with a margin and a floor, and its equation are removed. The notation table gains $\nu_t(k)$.

Other places that name the crowding cost

The loop section, the scoring section and the Background refer to "the congestion term K" of the storage objective. They would say "the crowding cost of congestion-aware storage (Section VI-A)" instead, so the reader is not sent to an equation in a later chapter.

Part 4. The code

For the thesis settings (`configs/step12`), checked in `storage/arrival.py` and `environment/multi_agent_env`

1. b_k **is real.** The SKUs take 0.5, 1 or 2 units of room, in turn.
2. **Whole refills, unplaced refills, the leg weights, the ties and the refill arrivals** already work as Parts 2 and 3 describe. With these choices the thesis follows the code, so no rule changes and nothing has to be run again.
3. **Only comments in the code are out of date.** The top comment of `storage/arrival.py` still describes the trip from the nearest dock and the old put-away score, and uses the old names (receiving vertex, delivery vertex). These would be brought in line with the thesis, with no change to what the code does. This goes in a small pull request on `slap-mapd-coupling`, which you open.

Questions for you

1. Do Parts 2 and 3 go into the thesis as shown?
2. Should the code comments be updated as in Part 4, point 3?
3. Should the differences found be noted in the tracking issue #103, marked as fixed by the thesis side?